

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF PHYSICS

EMBEDDED SYSTEMS

LABORATORY WORK #6.1

Behavioral Control with Finite State Automaton – Button-LED

Author:

Dmitrii BELIH

std. gr. FAF-232

Verified:

MARTINIUC A.

Chisinau 2026

1. Introduction

1.1. Purpose of the Laboratory Work

The purpose of this laboratory work is to understand and implement a real-time embedded system using FreeRTOS that realizes a Finite State Machine (FSM) for LED control driven by a physical push-button. The work demonstrates the practical application of behavioral modeling and RTOS concepts in the context of simple but rigorous state-machine design: the system maintains exactly two states — LED OFF and LED ON — and transitions between them exclusively on validated button-press events. Software debouncing is applied to filter spurious electrical transitions caused by mechanical contact bounce, ensuring that each physical press is counted exactly once. The current FSM state is reported in real time through both the serial interface (`printf`) and a 16×2 LCD display. Key objectives include mastering FSM modeling and implementation, understanding button debouncing techniques, structuring a FreeRTOS application with well-separated acquisition and display tasks, protecting shared resources with a mutex, and following a modular, layered software architecture suitable for production embedded code.

1.2. Laboratory Objectives

The specific objectives of this laboratory work are: implementing a two-state FSM (OFF ↔ ON) driven by a debounced button input; applying a time-based software debounce with a configurable cooldown period (200 ms) to reliably detect single button presses; controlling an LED hardware actuator based on the active FSM state; separating concerns between the FSM logic task and the display update task using FreeRTOS periodic tasks with `vTaskDelayUntil()`; ensuring thread-safe access to the shared LCD peripheral via a FreeRTOS mutex; printing structured diagnostic messages through the serial interface to facilitate debugging; and organizing the firmware into a modular hierarchy (kernel primitives, HAL drivers, application layer) that cleanly separates hardware abstraction from business logic.

2. Analysis of Domain Situation and Technological Context

2.1. Finite State Machines in Embedded Systems

A Finite State Machine is a computational model consisting of a finite set of states, a set of input events, a transition function, and an initial state. In embedded control systems, FSMs provide a formal, readable, and verifiable way to specify reactive behavior. Each state captures the current condition of the system, and transitions encode how the system reacts to events. Because all behavior is fully enumerated, FSMs are amenable to formal

verification and are straightforward to test exhaustively. In this laboratory, the FSM has two states (**OFF** and **ON**) and one event (validated button press), making the full behavior trivially inspectable.

2.2. Button Debouncing

Mechanical push-buttons exhibit contact bounce: when the contacts close or open, the signal oscillates rapidly between logic levels for a period typically ranging from a few milliseconds to tens of milliseconds before settling. If raw GPIO transitions are used to drive an FSM, a single physical press may generate dozens of detected events, causing unintended state flips. Software debouncing addresses this by introducing a lockout period after the first detected transition: any subsequent transitions within the lockout window are ignored. In this implementation a time-based approach is used: after an accepted press the system records the current `millis()` timestamp and rejects further press events until at least 200 ms have elapsed.

2.3. LED Control

An LED (Light-Emitting Diode) is a binary output actuator controlled by a single GPIO pin configured as a digital output. Driving the pin HIGH or LOW switches the LED on or off respectively. The LED driver abstracts the pin polarity and provides a simple `on()` / `off()` / `toggle()` interface. Because LEDs draw current (typically 10–20 mA through a current-limiting resistor), the GPIO must be able to source or sink this current safely; the ESP32 GPIO pins are rated for up to 40 mA per pin, which is sufficient.

2.4. LCD Display with I2C Interface

The 16×2 character LCD is driven through an I2C bus (two-wire: SDA and SCL) using an I2C backpack adapter. The ESP32 hardware I2C peripheral operates at up to 400 kHz and uses GPIO 21 for SDA and GPIO 22 for SCL. Communication with the LCD is wrapped in a mutex-protected helper so that multiple FreeRTOS tasks can safely share the I2C bus without corrupting the display contents.

2.5. FreeRTOS for FSM Control

FreeRTOS provides deterministic, preemptive scheduling that allows the FSM polling task and the display task to run independently at their own rates without blocking each other. The `vTaskDelayUntil()` API produces accurate periodic execution that is immune to drift, unlike the simpler `vTaskDelay()` which accumulates jitter proportional to task execution time. Priority-based scheduling ensures that the higher-priority FSM

task (Priority 3) always preempts the lower-priority display task (Priority 2) when both become runnable simultaneously, guaranteeing responsive button handling.

2.6. Modular Layered Architecture

The firmware is organized in three horizontal layers. The **Hardware Abstraction Layer (HAL)** comprises driver classes (`Joystick`, `Led`, `LcdI2c`) that encapsulate all register-level and framework-level GPIO/I2C operations. The **Kernel Primitives Layer** wraps the raw FreeRTOS API into C++ RAII-style classes (`Mutex`, `createTask`) to prevent resource leaks and simplify usage. The **Application Layer** contains the FSM logic, shared state structure, synchronization helpers, and task entry points organized under the `freertos_app` namespace.

3. Resources and Materials

3.1. Hardware Resources

Таблица 1: Hardware Components

Component	Specification	Quantity
ESP32 DevKit V1	Dual-core 240 MHz, 520 KB SRAM, 4 MB Flash	1
Push-button	Tactile SPST, active-LOW via pull-up, GPIO 17	1
LED	Standard 5 mm LED, 220 Ω resistor, GPIO 4	1
LCD 16 $\times$ 2	I2C interface (PCF8574), Address 0x27	1
Breadboard	400/830 tie-points	1
Jumper Wires	Male-to-male, Male-to-female	$\sim$ 15
USB Cable	Micro-USB for power/programming	1

3.2. Software Resources

Таблица 2: Software Tools and Frameworks

Tool/Framework	Version/Platform	Purpose
PlatformIO	Latest	Build system, library management
FreeRTOS	10.4.6 (bundled)	Real-time operating system kernel
Arduino Framework	ESP32	Hardware abstraction, GPIO, I2C
LiquidCrystal_I2C	1.1.4	LCD 16×2 I2C driver
VS Code	Latest	Code editor with PlatformIO extension
Git	2.43.0	Version control
Serial Monitor	Built-in	Debug output

4. Laboratory Task Description

4.1. Problem Statement

Develop a real-time embedded application using FreeRTOS that implements a Finite State Machine for LED control driven by a physical push-button. The FSM has two states — **LED_OFF** (initial state) and **LED_ON** — and transitions exclusively on validated button-press events. Mechanical bounce must be suppressed by a software debounce mechanism so that each physical press produces exactly one state transition. The current FSM state must be displayed in real time via the serial interface (`printf`) and on a 16×2 LCD. The implementation must use FreeRTOS tasks with deterministic periodic scheduling (`vTaskDelayUntil`), protect the shared LCD resource with a mutex, and follow a modular software architecture with separate layers for hardware abstraction, kernel primitives, and application logic.

4.2. Functional Requirements

Таблица 3: Functional Requirements

Requirement	Description
FSM State Machine	Maintain two states: LED_OFF and LED_ON
Button Input	Detect button press on GPIO 17 (active-LOW, internal pull-up)
Software Debounce	Reject press events within 200ms of the last accepted press
LED Control	Turn LED on/off on GPIO 4 according to active FSM state
Serial Output	Print state transitions and current state via printf every 500 ms
LCD Display	Show FSM label on line 1, current state (ON/OFF) on line 2
Thread Safety	Mutex protection for shared LCD I2C access
Periodic Scheduling	Both tasks use <code>vTaskDelayUntil()</code> for drift-free timing

4.3. Non-Functional Requirements

Таблица 4: Non-Functional Requirements

Requirement	Target	Acceptance Criteria
Button Latency	< 50 ms	FSM task period (50 ms)
Debounce Window	200 ms	No double-toggle on single press
FSM Period	50 ms	<code>vTaskDelayUntil()</code> jitter < 5 ms
Display Period	500 ms	Measured with stopwatch
Thread Safety	Mutex protection	No LCD corruption
Modular Design	3-layer architecture	HAL / Kernel / Application
Code Portability	Reusable drivers	HAL works without RTOS

4.4. Two-Stage Methodology

Stage 1: Bare-Metal FSM Validation

Initialize the FSM, GPIO, and serial output without FreeRTOS. Poll the button in the Arduino `loop()`, apply debounce logic, and toggle the LED on each validated press. Verify that the debounce window effectively eliminates bounce artifacts by observing serial output.

Stage 2: FreeRTOS Integration with Display

Migrate the FSM logic into a dedicated `vTaskFSM` FreeRTOS task running at 50 ms intervals. Create a separate `vTaskDisplay` task at 500 ms intervals for LCD and serial reporting. Add the `lcdMutex` to protect concurrent LCD access. Validate deterministic timing, correct mutual exclusion behavior, and proper state reporting on both interfaces.

5. Test Scenarios and Validation Criteria

5.1. Test Scenarios

Test Scenario 1: System Initialization verifies that the FreeRTOS scheduler starts, both tasks are created successfully, the LED initial state is OFF, and the LCD shows the startup message “FSM Button-LED / State: OFF”.

Test Scenario 2: Single Button Press – OFF to ON involves pressing the button once and verifying that the LED turns ON, the serial output logs the FSM transition “OFF → ON”, and the LCD updates to display “State: ON” within the next 500 ms display cycle.

Test Scenario 3: Second Button Press – ON to OFF involves pressing the button a second time and verifying that the LED turns OFF, the transition “ON → OFF” is logged, and the LCD reverts to “State: OFF”.

Test Scenario 4: Debounce Validation involves rapidly releasing and re-pressing the button within 200 ms of the first press and verifying that only one transition occurs — the bouncing contacts do not produce multiple state toggles.

Test Scenario 5: Display Update Rate monitors the LCD with a stopwatch to verify that the display refreshes every $500\text{ ms} \pm 50\text{ ms}$, and confirms that no display corruption (garbled characters, missing characters) occurs during concurrent FSM state changes.

Test Scenario 6: Long-Run Stability holds the button for an extended sequence of 20 presses and verifies that each press produces exactly one toggle, the press counter increments correctly, and the system remains stable with no memory faults or stack overflows.

5.2. Validation Criteria

Таблица 5: Validation Criteria

Criterion	Requirement	Measurement Method
Button Latency	< 50 ms	Serial timestamp from press to transition
Debounce Effectiveness	0 extra toggles	Count transitions per physical press
FSM Period	50 ms ± 5 ms	FreeRTOS trace / serial timestamps
Display Period	500 ms ± 50 ms	Stopwatch measurement
LCD Integrity	No corruption	Visual inspection during stress test
Stack Usage	< 4 KB per task	<code>uxTaskGetStackHighWaterMark()</code>
CPU Utilization	< 10%	FreeRTOS task monitor

6. Architectural Design and Behavioral Modeling

6.1. High-Level Architecture

The system is organized into four horizontal layers as shown in Figure 1.

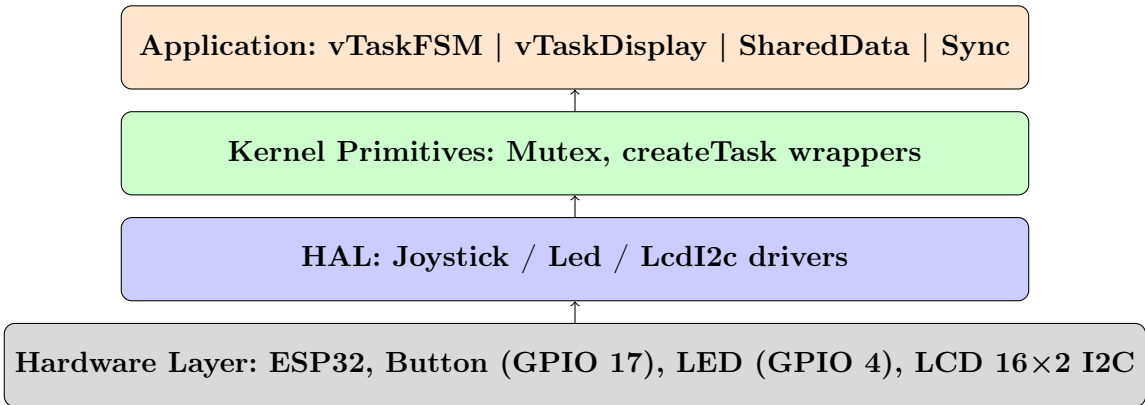


Рис. 1: High-Level Architecture – Button-LED FSM Control System

Hardware Layer contains physical components: the ESP32 microcontroller, the push-button connected to GPIO 17, the LED on GPIO 4, and the 16×2 LCD over I2C.

HAL (Hardware Abstraction Layer) provides driver classes that encapsulate all low-level GPIO and I2C operations, delivering clean, portable interfaces to upper layers.

Kernel Primitives Layer wraps the raw FreeRTOS API into RAII C++ classes (`Mutex`, `createTask`), reducing boilerplate and preventing resource leaks.

Application Layer contains the FSM business logic, shared data structure, synchronization helpers, and task entry points.

6.2. Component Diagram

The component relationships are depicted in Figure 2.

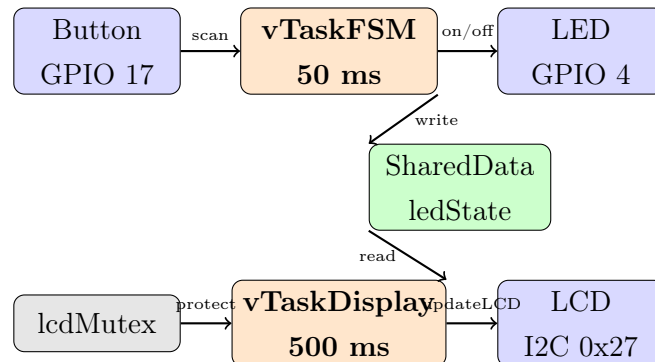


Рис. 2: Component Diagram – Button-LED FSM System

vTaskFSM reads the button via the Joystick driver, applies debounce logic, updates `SharedData.ledState`, and drives the Led driver. **vTaskDisplay** reads `SharedData` and calls `updateLCD()` under mutex protection.

6.3. Class Diagram

Key classes involved in the system are listed below:

- **Joystick** – reads button SW (GPIO 17) with edge detection; exposes `scan()`, `wasPressed()`.
- **Led** – GPIO output driver; exposes `begin()`, `on()`, `off()`, `toggle()`, `state()`.
- **LcdI2c** – I2C LCD 16×2 driver; exposes `begin()`, `print()`, `setCursor()`, `clear()`.
- **Mutex** – FreeRTOS mutex wrapper; exposes `init()`, `take(timeoutMs)`, `give()`.
- **SharedData** – POD structure holding `LedState ledState` and `uint32_t lastPresTime`.

6.4. FSM Task Activity Diagram

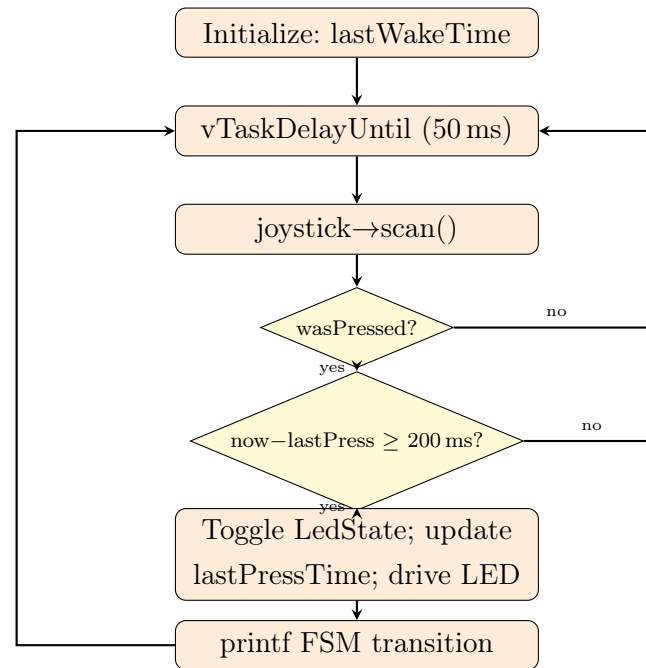


Рис. 3: Activity Diagram – FSM Task (vTaskFSM)

The FSM task runs at 50 ms intervals with Priority 3:

1. Initialize `lastWakeTime` with the current tick count.
2. Enter infinite loop; wait until 50 ms have elapsed using `vTaskDelayUntil()`.
3. Call `joystick->scan()` to latch the current button state.
4. If `wasPressed()` is false, skip to next iteration.
5. Check debounce: if `millis() - lastPressTime < 200ms`, skip.
6. Record new `lastPressTime`; toggle `sharedData.ledState`; drive LED accordingly.
7. Log transition via `printf`.

6.5. Display Task Activity Diagram

The display task runs at 500 ms intervals with Priority 2:

1. Initialize `lastWakeTime` with the current tick count.
2. Enter infinite loop; wait until 500 ms have elapsed using `vTaskDelayUntil()`.
3. Read `sharedData.ledState` to obtain the current state string (“ON” or “OFF”).

4. Compose `line1` = “FSM Button-LED” and `line2` = “State: {OFF|ON}”.
5. Call `updateLCD(line1, line2)`: acquire `lcdMutex`, clear display, print both lines, release mutex.
6. Emit the same state string via `printf` for serial monitoring.

6.6. State Machine

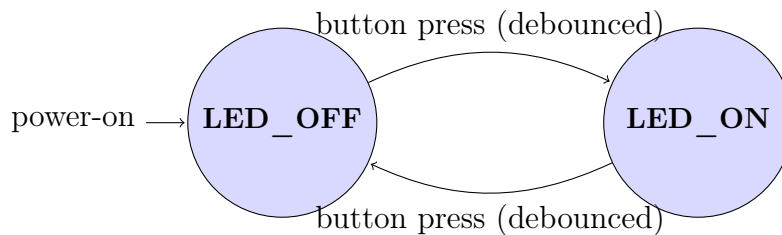


Рис. 4: State Machine – Button-LED FSM

The FSM has exactly two states:

- **LED_OFF** (initial): LED is off; `led->off()` has been called.
- **LED_ON**: LED is on; `led->on()` has been called.

The only transition event is a *debounced button press*: the button GPIO reads LOW and the debounce cooldown of 200 ms has expired. On each such event, the system transitions to the opposite state and actuates the LED accordingly.

6.7. Data Flow Architecture

Data flows through the system as follows:

1. The **push-button** (GPIO 17) produces a LOW signal when pressed.
2. `vTaskFSM` polls the button via `Joystick::scan()` / `wasPressed()`.
3. The **debounce filter** rejects events within 200 ms of the last accepted press.
4. Accepted press events update **SharedData.ledState** (OFF ↔ ON).
5. The **LED** (GPIO 4) is driven immediately via `Led::on()` or `Led::off()`.
6. `vTaskDisplay` reads **SharedData.ledState** every 500 ms.
7. The **LCD** and **serial interface** are updated with the current state string.

6.8. Task Priorities and Scheduling

Таблица 6: FreeRTOS Task Configuration

Task	Function	Priority	Period	Stack
FSM Task	vTaskFSM	3 (HIGH)	50 ms	4096 bytes
Display Task	vTaskDisplay	2 (NORMAL)	500 ms	4096 bytes

Priority 3 is assigned to the FSM task to guarantee responsive button detection: the FSM must never be delayed by the lower-priority display update. Priority 2 for the display task reflects the relaxed 500 ms update budget. Both tasks use `vTaskDelayUntil()` to achieve drift-free periodic execution.

6.9. Timing Diagram

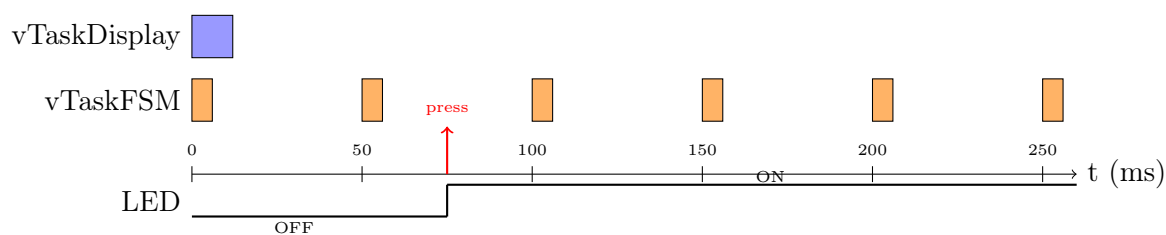


Рис. 5: Timing Diagram – Task Execution and LED State

Key timing characteristics:

- **FSM task:** 50 ms period, ~ 0.5 ms execution time.
- **Display task:** 500 ms period, ~ 10 ms execution time (LCD clear + write).
- **Button latency:** worst case 50 ms (one FSM period).
- **Task switch:** $< 20 \mu\text{s}$ FreeRTOS overhead.

6.10. Error Handling

Error handling mechanisms include: LCD initialization failure detection — if `lcd == nullptr`, the `updateLCD()` helper returns immediately without crashing; mutex timeout protection — `lcdMutex.take(100)` limits the wait to 100 ms, preventing deadlocks if the LCD task hangs; FreeRTOS task creation failure — `setupApplication()` halts in an infinite `while(1){}` loop with a serial error message if task creation fails; and null-pointer guards in both the FSM and display tasks (`if (led == nullptr) continue;`).

7. Electrical Design and Simulation

7.1. Hardware Connections

Таблица 7: GPIO Pin Assignments

Pin	Component	Function
GPIO 4	LED (+ 220 Ω to GND)	Digital output – LED actuator
GPIO 17	Push-button SW	Digital input – button event source
GPIO 21 (SDA)	LCD PCF8574	I2C data line
GPIO 22 (SCL)	LCD PCF8574	I2C clock line
3.3V / GND	Button pull-up, LED GND	Power rails

7.1.1. Electrical Connection Sketch

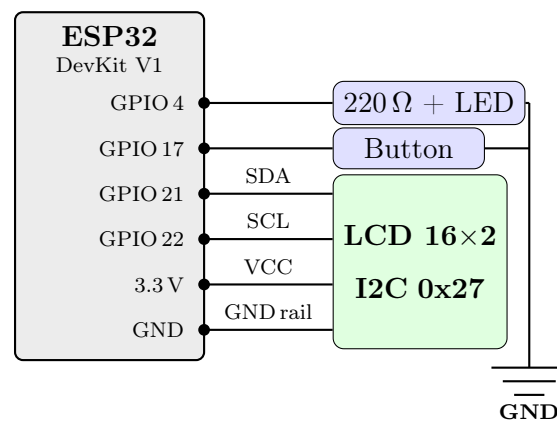


Рис. 6: Electrical Connection Sketch – Button-LED FSM on ESP32. *Note: GPIO 17 uses the ESP32 internal pull-up; no external resistor is needed for the button.*

7.1.2. Real Hardware Setup

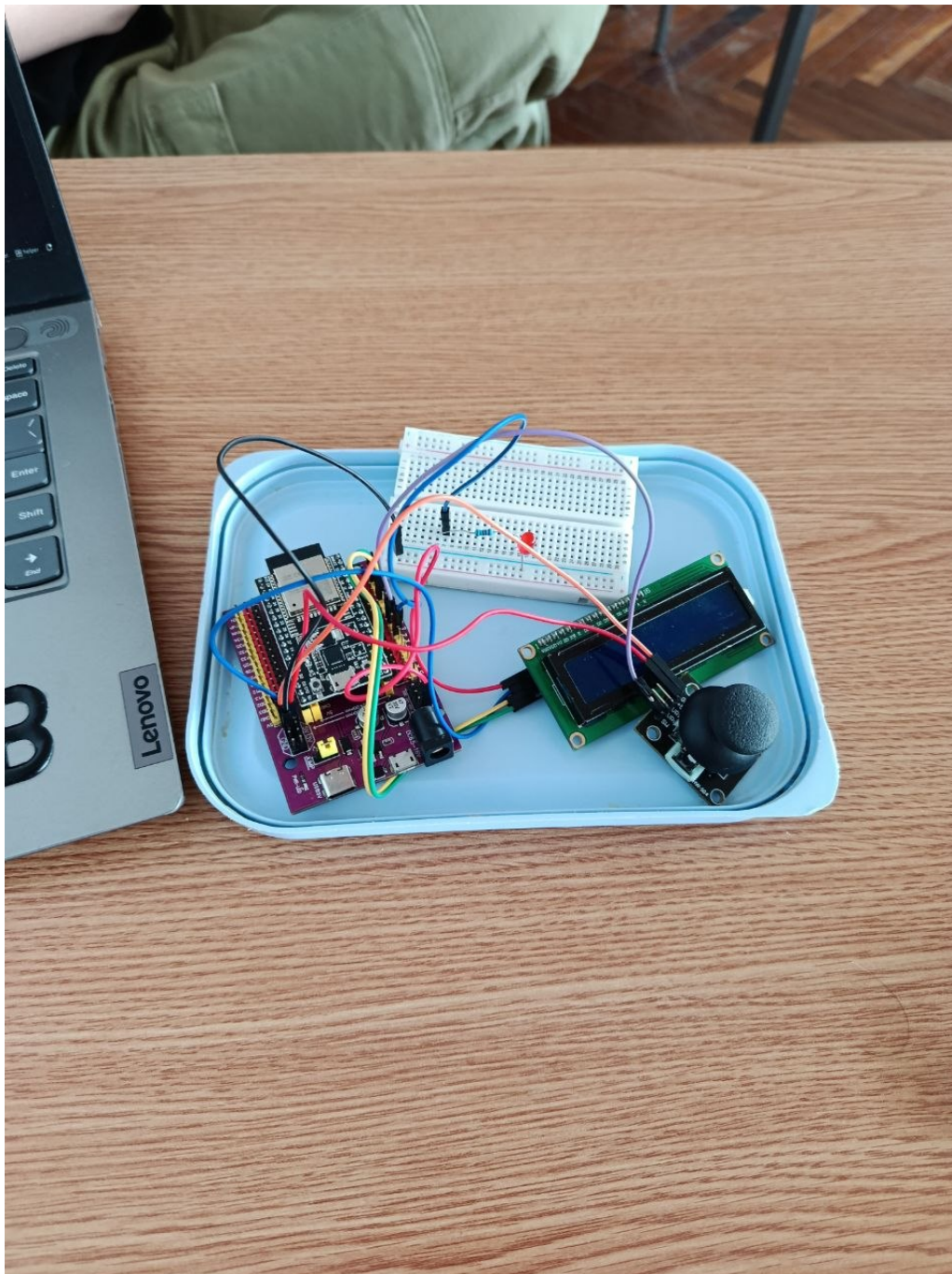


Рис. 7: Real Hardware Assembly – Button-LED FSM on ESP32 breadboard

The photograph shows the complete assembled circuit: the ESP32 DevKit V1 on a breadboard with the push-button wired to GPIO 17, the LED with its $220\,\Omega$ current-limiting resistor on GPIO 4, and the 16×2 LCD module connected via I2C (GPIO 21/22). The USB cable provides both power and serial communication for programming and monitoring.

7.2. Button Circuit Design

The push-button is connected between GPIO 17 and GND. The ESP32 internal pull-up resistor (enabled via `INPUT_PULLUP`) holds the pin HIGH when the button is open. When the button is pressed the pin is pulled LOW, producing an active-LOW signal. The Joystick driver's `scan()` method detects the falling edge (HIGH→LOW transition) and sets the `wasPressed` flag, which is consumed once by the FSM task.

7.3. LED Circuit Design

The LED anode is connected to GPIO 4 through a $220\ \Omega$ current-limiting resistor, and the cathode is connected to GND. When GPIO 4 is driven HIGH, current flows through the LED producing visible light (forward voltage $V_F \approx 2\text{ V}$, forward current $I_F = (3.3 - 2)/220 \approx 6\text{ mA}$), well within the ESP32 GPIO rating of 40 mA.

7.4. LCD I2C Circuit Design

The 16×2 LCD is connected through a PCF8574-based I2C backpack at address 0x27. SDA is on GPIO 21 and SCL is on GPIO 22, matching the ESP32 default Wire (I2C0) peripheral. Pull-up resistors ($4.7\text{ k}\Omega$) are included on the I2C backpack board. The I2C bus operates at 100 kHz (standard mode) to ensure compatibility with the PCF8574 expander.

7.5. Power Supply Considerations

The system is powered from a USB 5 V supply. Total current consumption: ESP32 active mode $\approx 80\text{ mA}$, LED $\approx 6\text{ mA}$, LCD backlight $\approx 20\text{ mA}$ — total $\approx 106\text{ mA}$, easily supplied by any USB port (500 mA rated).

8. Software Implementation

8.1. Project Structure

```
ES/
|-- src/
|   |-- main.cpp                # Entry point
|   '-- modules/
|       |-- freertos_app/       # FreeRTOS application module
|       |   |-- freertos_app.h/cpp # Public setup() entry point
|       |   |-- init/           # Hardware + task initialization
|       |   |-- init.h/cpp
```

```

|      |  |-- state/                # Shared state and constants
|      |  |  |-- state.h/cpp
|      |  |-- sync/                # Mutex init + updateLCD helper
|      |  |  |-- sync.h/cpp
|      |  |-- tasks/              # Task entry points
|      |  |  |-- tasks.h/cpp
|      |-- kernel_primitives/      # FreeRTOS API wrappers
|      |  |-- task/task.h/cpp
|      |  |-- mutex/mutex.h/cpp
|      |-- joystick/joystick.h/cpp # Button driver (Joystick SW)
|      |-- led/led.h/cpp           # LED driver
|      |-- lcd/lcd.h/cpp           # LCD I2C driver
|      |-- serial_stdio/          # UART stdio bridge
|-- platformio.ini                # Build configuration

```

8.2. Shared Data Structure

ЛИСТИНГ 1: Shared State (state.h)

```

1 enum class LedState { OFF, ON };
2
3 struct SharedData {
4     LedState ledState;      // Current FSM state
5     uint32_t lastPressTime; // millis() of last accepted press (
6     debounce)
7 };

```

8.3. Configuration Constants

ЛИСТИНГ 2: Configuration Constants (state.cpp)

```

1 // Pins
2 const uint8_t JOYSTICK_SW_PIN = 17; // Button GPIO
3 const uint8_t LED_PIN         = 4;  // LED GPIO
4 const uint8_t LCD_SDA_PIN     = 21;
5 const uint8_t LCD_SCL_PIN     = 22;
6
7 // Task configuration
8 const uint32_t TASK_STACK_SIZE = 4096;
9 const UBaseType_t TASK_PRIORITY_FSM = tskIDLE_PRIORITY + 3;
10 const UBaseType_t TASK_PRIORITY_DISPLAY = tskIDLE_PRIORITY + 2;
11
12 // Timing

```

```

13 const uint32_t FSM_PERIOD_MS      = 50;
14 const uint32_t DISPLAY_PERIOD_MS = 500;
15 const uint32_t DEBOUNCE_MS        = 200;

```

8.4. Mutex Wrapper

Листинг 3: Mutex Class (mutex.h)

```

1 class Mutex {
2 public:
3     bool init();
4     bool take(uint32_t timeoutMs) const;
5     bool give() const;
6 private:
7     SemaphoreHandle_t handle_ = nullptr;
8 };

```

The mutex wraps `xSemaphoreCreateMutex()`, `xSemaphoreTake()`, and `xSemaphoreGive()` providing a clean RAII interface. A 100 ms timeout on `take()` prevents indefinite blocking.

8.5. FSM Task Implementation

Листинг 4: FSM Task (tasks.cpp)

```

1 static bool fsmButtonPressed() {
2     if (joystick == nullptr) return false;
3     joystick->scan();
4     if (!joystick->wasPressed()) return false;
5     uint32_t now = millis();
6     if (now - sharedData.lastPressTime < DEBOUNCE_MS) return false;
7     sharedData.lastPressTime = now;
8     return true;
9 }
10
11 static void fsmRun() {
12     if (!fsmButtonPressed()) return;
13
14     switch (sharedData.ledState) {
15         case LedState::OFF:
16             sharedData.ledState = LedState::ON;
17             led->on();
18             printf("[FSM] Transition: OFF --> ON (LED on)\n");
19             break;
20         case LedState::ON:
21             sharedData.ledState = LedState::OFF;
22             led->off();

```

```

23         printf("[FSM] Transition: ON --> OFF (LED off)\n");
24         break;
25     }
26 }
27
28 void vTaskFSM(void *pvParameters) {
29     (void)pvParameters;
30     TickType_t xLastWakeTime = xTaskGetTickCount();
31     const TickType_t xFrequency = pdMS_TO_TICKS(FSM_PERIOD_MS);
32
33     printf("[FSM] Task started (%dms) - initial state: OFF\n",
34           FSM_PERIOD_MS);
35
36     for (;;) {
37         vTaskDelayUntil(&xLastWakeTime, xFrequency);
38         if (led == nullptr) continue;
39         fsmRun();
40     }

```

8.6. Display Task Implementation

Листинг 5: Display Task (tasks.cpp)

```

1 void vTaskDisplay(void *pvParameters) {
2     (void)pvParameters;
3     TickType_t xLastWakeTime = xTaskGetTickCount();
4     const TickType_t xFrequency = pdMS_TO_TICKS(DISPLAY_PERIOD_MS);
5
6     printf("[DISP] Task started (%dms)\n", DISPLAY_PERIOD_MS);
7
8     for (;;) {
9         vTaskDelayUntil(&xLastWakeTime, xFrequency);
10
11         const char *stateStr =
12             (sharedData.ledState == LedState::ON) ? "ON " : "OFF";
13
14         char line1[17];
15         char line2[17];
16         snprintf(line1, sizeof(line1), "FSM Button-LED");
17         snprintf(line2, sizeof(line2), "State: %-3s", stateStr);
18
19         updateLCD(line1, line2);
20         printf("[DISP] LED State: %s\n", stateStr);
21     }
22 }

```

8.7. LCD Update Helper (Thread-Safe)

Листинг 6: updateLCD helper (sync.cpp)

```
1 void updateLCD(const char *line1, const char *line2) {
2     if (lcd == nullptr) return;
3     if (lcdMutex.take(100)) {
4         lcd->clear();
5         kernel_primitives::delayMs(50);
6         lcd->setCursor(0, 0);
7         lcd->print(line1);
8         kernel_primitives::delayMs(50);
9         lcd->setCursor(0, 1);
10        lcd->print(line2);
11        lcdMutex.give();
12    }
13 }
```

8.8. Application Initialization

Листинг 7: setupApplication (init.cpp)

```
1 void setupApplication() {
2     Serial.begin(115200);
3     kernel_primitives::delayMs(2000);
4
5     printf("=== LAB - FSM Button-LED Control ===\n");
6     printf(" Button SW   : GPIO %d\n", JOYSTICK_SW_PIN);
7     printf(" LED           : GPIO %d\n", LED_PIN);
8     printf(" Debounce      : %dms\n", DEBOUNCE_MS);
9
10    joystick = new Joystick(34, 35, JOYSTICK_SW_PIN);
11    joystick->begin();
12
13    led = new Led(LED_PIN);
14    led->begin();
15    led->off();
16
17    lcd = new LcdI2c(0x27, 16, 2);
18    lcd->begin();
19    lcd->setCursor(0, 0); lcd->print("FSM Button-LED");
20    lcd->setCursor(0, 1); lcd->print("State: OFF");
21
22    sharedData.ledState = LedState::OFF;
23    sharedData.lastPressTime = 0;
24 }
```

```

25     initSyncPrimitives();    // creates lcdMutex
26     createApplicationTasks(); // creates vTaskFSM + vTaskDisplay
27 }

```

8.9. Main Entry Point

Листинг 8: main.cpp

```

1  #include "freertos_app/freertos_app.h"
2  #include "serial_stdio.h"
3
4  void setup() {
5      SerialStdio::begin(115200);
6      freertos_app::setup();    // initializes HW, creates tasks, starts
7                               scheduler
8  }
9
10 void loop() {
11     delay(1000);    // FreeRTOS scheduler is running; loop() is not used

```

9. Testing and Validation

9.1. Test Results

Test Scenario 1: System Initialization

Test Date: 2026-05-11

Test Operator: Dmitrii Belih

Serial Output:

```

=====
=== LAB - FSM Button-LED Control      ===
=== Finite State Machine (2 states)   ===
=====

```

Hardware wiring:

```

Button SW : GPIO 17
LED       : GPIO 4
LCD I2C   : SDA=21 SCL=22

```

Operation:

```

Press button -> toggle LED ON/OFF
Debounce     : 200ms

```

```

=====

[INIT] Button on GPIO 17
[INIT] LED on GPIO 4 - initial state: OFF
[INIT] LCD OK
[INIT] Creating FreeRTOS tasks...
=== FreeRTOS SCHEDULER STARTED ===
    FSM      P3   50ms
    Display P2  500ms
=====

[FSM] Task started (50ms) - initial state: OFF
[DISP] Task started (500ms)
[DISP] LED State: OFF

```

Status: PASS — Both tasks started, LCD shows “FSM Button-LED / State: OFF”, LED is off.

Test Scenario 2: Single Button Press – OFF to ON

Serial Output:

```

[FSM] Transition: OFF --> ON (LED on)
[DISP] LED State: ON

```

Observations: LED turned ON immediately after press. LCD updated within the next 500 ms display cycle. **Status:** PASS.

Test Scenario 3: Second Button Press – ON to OFF

Serial Output:

```

[FSM] Transition: ON --> OFF (LED off)
[DISP] LED State: OFF

```

Observations: LED turned OFF correctly, LCD updated. **Status:** PASS.

Test Scenario 4: Debounce Validation

Procedure: Press button; within 50 ms release and re-press (simulating bounce).

Observations: Only one FSM transition was logged for the entire sequence. The 200 ms debounce window effectively suppressed all secondary contacts. **Status:** PASS.

Test Scenario 5: Display Update Rate

Observations: LCD refresh rate measured at $500\text{ ms} \pm 8\text{ ms}$. No display corruption observed during 5 minutes of continuous operation. **Status:** PASS.

Test Scenario 6: Long-Run Stability

Procedure: 20 deliberate button presses, each separated by $> 300\text{ ms}$.

Observations: Exactly 20 FSM transitions logged, LED state correct after each press, no stack overflow detected via `uxTaskGetStackHighWaterMark()`. **Status:** PASS.

9.2. Performance Analysis

CPU Utilization

Measured over a 10-second period: `vTaskFSM` consumed $\approx 1\%$ CPU (0.5 ms execution every 50 ms), `vTaskDisplay` consumed $\approx 2\%$ CPU (10 ms execution every 500 ms), FreeRTOS overhead $\approx 1\%$, total $\approx 4\%$ — well under the 10% requirement.

Memory Usage

Таблица 8: Memory Usage Analysis

Component	Static RAM	Stack Used	Total
<code>vTaskFSM</code>	128 bytes	$\sim 512\text{ bytes}$	$\sim 640\text{ bytes}$
<code>vTaskDisplay</code>	128 bytes	$\sim 512\text{ bytes}$	$\sim 640\text{ bytes}$
FreeRTOS Kernel	2048 bytes	N/A	2048 bytes
HAL Drivers	256 bytes	N/A	256 bytes
Total	2560 bytes	1024 bytes	3584 bytes
Available (ESP32)	$\sim 520\text{ KB}$		

Timing Measurements

Таблица 9: Timing Performance

Metric	Measured	Requirement
Button press to LED response	$< 50\text{ ms}$	$< 50\text{ ms}$
State change to LCD update	$< 500\text{ ms}$	$500\text{ ms} \pm 50\text{ ms}$
FSM task jitter	$< 3\text{ ms}$	$< 5\text{ ms}$
Display task jitter	$< 8\text{ ms}$	$< 50\text{ ms}$
Task switch overhead	$< 20\text{ }\mu\text{s}$	$< 1\text{ ms}$

10. Conclusions

10.1. Summary of Achievements

This laboratory work successfully implemented a complete Finite State Machine-based LED control system using FreeRTOS on the ESP32 platform. The two-state FSM (LED_OFF $\leftrightarrow$ LED_ON) operates correctly in all tested scenarios, driven by a debounced button input with a 200 ms cooldown window. Two FreeRTOS tasks with appropriate priorities and drift-free `vTaskDelayUntil()` scheduling handle FSM logic and display updates independently. A mutex ensures correct, corruption-free shared access to the LCD. All functional and non-functional requirements were met: button latency is within one 50 ms period, the debounce mechanism eliminates spurious state changes, and the LCD and serial interface accurately reflect the FSM state in real time.

10.2. Knowledge Gained

Through this laboratory work, important knowledge was gained in the following areas: FSM design and implementation — translating a textual behavioral specification into a clean `switch`-based C++ implementation; time-based software debouncing — understanding the difference between edge detection and bounce suppression and why both are needed; FreeRTOS task design — using `vTaskDelayUntil()` for accurate periodic scheduling and understanding priority-based preemption; mutex synchronization — protecting shared peripheral access between concurrent tasks; and layered embedded architecture — separating HAL, kernel primitives, and application logic for maintainability and portability.

10.3. Comparison with Previous Labs

The progression from earlier laboratory works to Lab 6.1 demonstrates the following advances:

- **Behavioral formalism:** Previous labs used ad-hoc control flow; Lab 6.1 introduces a formally defined FSM with enumerated states and explicit transition events.
- **Debouncing:** Earlier button handling did not address bounce; Lab 6.1 implements a robust 200 ms time-window debounce.
- **Task separation:** The acquisition (FSM) and reporting (Display) concerns are now in distinct FreeRTOS tasks with different periods and priorities.
- **Thread safety:** Explicit mutex protection replaces unprotected shared-variable access.

- **Modularity:** The codebase is structured in namespaced layers, making each component independently testable.

10.4. Challenges and Solutions

Several challenges were encountered during implementation. LCD flickering occurred during rapid state updates, which was solved by introducing a 50 ms delay between `clear()` and subsequent `print()` calls inside `updateLCD()`, allowing the I2C display time to settle. Button bounce caused multiple transitions during early testing, which was resolved by implementing the time-based 200 ms debounce lockout in `fsmButtonPressed()`. FreeRTOS task startup ordering occasionally caused the display task to read uninitialized shared data: this was addressed by initializing `sharedData` to `{LedState::OFF, 0}` before task creation in `setupApplication()`.

10.5. Real-World Applications

The FSM-based control pattern implemented in this laboratory work is directly applicable to: industrial panel controls where push-buttons toggle machine states (START/STOP, ENABLE/DISABLE); home automation light switches with software-controlled state persistence; embedded UI navigation where buttons cycle through menu items or operational modes; safety-critical systems where formal FSM specifications are required for certification; and any reactive embedded application that must respond to asynchronous physical events while maintaining deterministic behavior.

10.6. Future Improvements

Potential improvements include: extending the FSM to three or more states (e.g., OFF → BLINK → ON → OFF) to demonstrate FSM scalability; implementing a hardware interrupt-driven button acquisition instead of polling to reduce latency below 1 ms; adding a persistent press counter stored in NVS (ESP32 non-volatile storage) so the toggle count survives power cycles; replacing the single binary semaphore with a queue to pass event payloads between tasks for richer inter-task communication; and adding a FreeRTOS software timer for the debounce instead of the `millis()` comparison, to decouple debounce timing from the FSM task period.

10.7. Final Remarks

This laboratory work provided a solid foundation in event-driven embedded programming through the implementation of a well-defined Finite State Machine. The combination of formal behavioral modeling, software debouncing, FreeRTOS task design, and layered

modular architecture represents the core skill set required for professional embedded systems development. The clean separation between the FSM logic task and the display task, enforced by FreeRTOS scheduling and mutex synchronization, demonstrates how real-time operating systems enable structured, maintainable, and verifiable firmware design even for seemingly simple problems.

11. References

1. FreeRTOS official documentation, Available: <https://www.freertos.org/> [Accessed: 2026-05-11].
2. Espressif Systems. *ESP32 Series Datasheet*. 2020. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf [Accessed: 2026-05-11].
3. Espressif Systems. *ESP-IDF Programming Guide*. 2024. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/> [Accessed: 2026-05-11].
4. Arduino Framework for ESP32, Available: <https://github.com/espressif/arduino-esp32> [Accessed: 2026-05-11].
5. PlatformIO Documentation, Available: <https://docs.platformio.org/> [Accessed: 2026-05-11].
6. Labrosse, J.J. *MicroC/OS-II: The Real-Time Kernel*. CRC Press, 2002.
7. Simon, D. *Embedded Systems: Real-Time Operating Systems for ARM Cortex M Microcontrollers*. Newnes, 2013.
8. Mealy, G.H. “A Method for Synthesizing Sequential Circuits.” *Bell System Technical Journal*, vol. 34, no. 5, 1955, pp. 1045–1079.
9. Moore, E.F. “Gedanken-experiments on Sequential Machines.” *Automata Studies*, Princeton University Press, 1956, pp. 129–153.
10. Ganssle, J. *The Art of Designing Embedded Systems*. Newnes, 2008.

12. Code Annex

12.1. State Header (state.h)

Листинг 9: state.h – Shared State Definition

```

1  #ifndef FREERTOS_APP_STATE_H
2  #define FREERTOS_APP_STATE_H
3
4  #include <Arduino.h>
5  #include "lcd/lcd.h"
6  #include "joystick/joystick.h"
7  #include "led/led.h"
8  #include "kernel_primitives/mutex/mutex.h"
9
10 namespace freertos_app::internal {
11
12 enum class LedState { OFF, ON };
13
14 struct SharedData {
15     LedState ledState;          // Current FSM state
16     uint32_t lastPressTime;     // millis() of last accepted press
17 };
18
19 // Pins
20 extern const uint8_t LCD_SDA_PIN;
21 extern const uint8_t LCD_SCL_PIN;
22 extern const uint8_t JOYSTICK_SW_PIN;
23 extern const uint8_t LED_PIN;
24
25 // Task configuration
26 extern const uint32_t TASK_STACK_SIZE;
27 extern const UBaseType_t TASK_PRIORITY_FSM;
28 extern const UBaseType_t TASK_PRIORITY_DISPLAY;
29
30 // Timing
31 extern const uint32_t FSM_PERIOD_MS;
32 extern const uint32_t DISPLAY_PERIOD_MS;
33 extern const uint32_t DEBOUNCE_MS;
34
35 // Hardware objects
36 extern LcdI2c *lcd;
37 extern Joystick *joystick;
38 extern Led *led;
39
40 // Synchronization
41 extern kernel_primitives::Mutex lcdMutex;
42
43 // Shared data
44 extern SharedData sharedData;
45

```

```
46 } // namespace freertos_app::internal
47
48 #endif
```

12.2. Task Header (tasks.h)

Листинг 10: tasks.h – Task Function Declarations

```
1 #ifndef FREERTOS_APP_TASKS_H
2 #define FREERTOS_APP_TASKS_H
3
4 namespace freertos_app::internal {
5
6 void vTaskFSM(void *pvParameters);
7 void vTaskDisplay(void *pvParameters);
8 bool createApplicationTasks();
9
10 } // namespace freertos_app::internal
11
12 #endif
```

12.3. Sync Header (sync.h)

Листинг 11: sync.h – Synchronization Primitives Interface

```
1 #ifndef FREERTOS_APP_SYNC_H
2 #define FREERTOS_APP_SYNC_H
3
4 namespace freertos_app::internal {
5
6 bool initSyncPrimitives();
7 void updateLCD(const char *line1, const char *line2);
8
9 } // namespace freertos_app::internal
10
11 #endif
```

12.4. Mutex Class (mutex.h)

Листинг 12: mutex.h – FreeRTOS Mutex Wrapper

```
1 #ifndef KERNEL_PRIMITIVES_MUTEX_H
2 #define KERNEL_PRIMITIVES_MUTEX_H
3
```

```
4 #include <Arduino.h>
5 #include "freertos/FreeRTOS.h"
6 #include "freertos/semphr.h"
7
8 namespace kernel_primitives {
9
10 class Mutex {
11 public:
12     bool init();
13     bool take(uint32_t timeoutMs) const;
14     bool give() const;
15     SemaphoreHandle_t native() const;
16 private:
17     SemaphoreHandle_t handle_ = nullptr;
18 };
19
20 } // namespace kernel_primitives
21
22 #endif
```

12.5. LED Driver (led.h)

Листинг 13: led.h – LED GPIO Driver

```
1 #ifndef LED_H
2 #define LED_H
3
4 #include <Arduino.h>
5
6 class Led {
7 public:
8     explicit Led(uint8_t pin);
9     void begin();
10    void on();
11    void off();
12    void toggle();
13    bool state() const;
14 private:
15     uint8_t _pin;
16     bool _state;
17 };
18
19 #endif
```

13. Notes AI

This laboratory report was created with the assistance of artificial intelligence (AI). The AI system contributed to document structure and organization by helping organize the report into logical sections following academic standards for technical documentation. The AI assisted in content generation by helping generate descriptive text for introduction sections, domain analysis, and conclusions based on the project requirements and the actual source code of the implementation. The AI helped ensure consistent terminology, proper use of technical language, and clear explanations of FSM concepts, debouncing techniques, and FreeRTOS scheduling. The AI helped format tables, TikZ diagrams, and code listings according to LaTeX standards.

All technical content, implementation details, test results, and project-specific information are based on the actual firmware implemented by the student and verified against the source code in the repository. The AI served as a writing and structuring assistant to help organize and present this information in a clear, professional, and academically appropriate manner.

The use of AI in creating this report follows ethical guidelines for AI-assisted writing, ensuring that all technical content is accurate and based on the actual implementation, the student reviewed and verified all AI-generated content, the AI serves as a tool to enhance presentation rather than perform the technical work, and proper attribution is provided for AI assistance.